Advanced Algorithms Final Project - Flow!

Tommy Liu and Kimberly Lopez

April 2026

Rules of the Game

You are provided with a grid with colored dots. Your task is to connect all pairs of colored dots (where each connection is called a "flow") without crossing the flows. Let us start with some concrete examples!

Example. Flow!

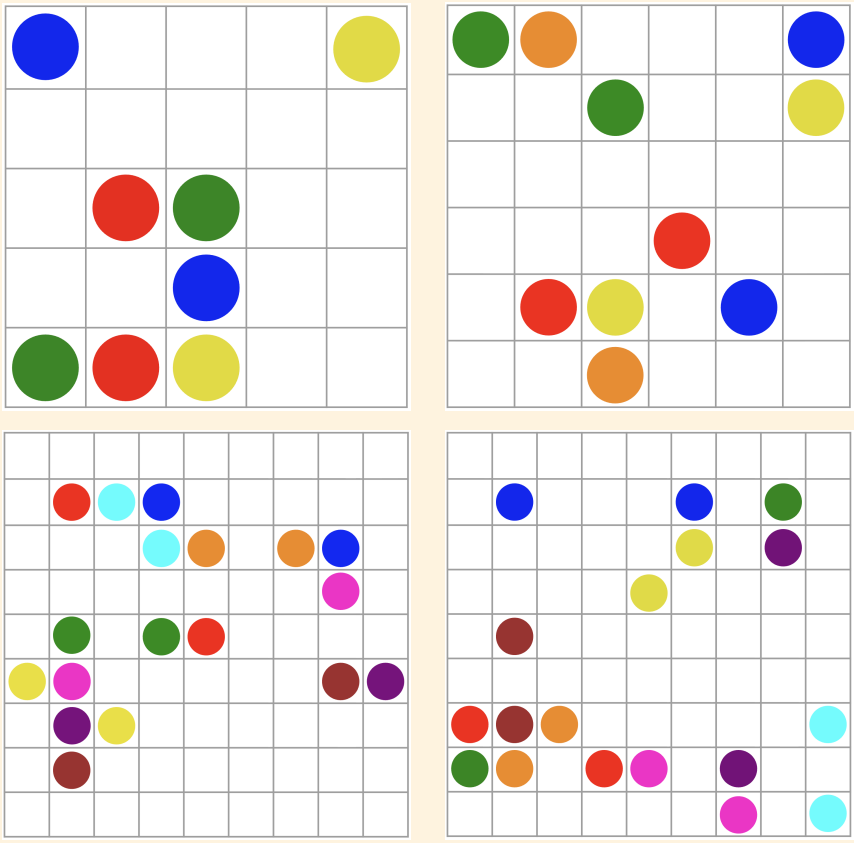


Figure 1: Four instances of the flow puzzle, with difficulty increasing from left to right and top to bottom. See if you can solve them!

Motivation

As you probably realized, the solution becomes less obvious as the grid grows larger. This project makes use of two linear programming strategies to solve the puzzle: Connectivity and Multi-Commodity Flow!

Formulation 1: Connectivity

We try to solve the problem via connectivity. For this formulation, we index over three sets: `ROWS`, `COLUMNS`, and `COLORS`. As with any linear program, we now need to define the **Decision Variables**, **Objective Function**, and **Constraints**.

Connectivity. Decision Variables

Let $x_{i,j,k}$ be the binary variable whose value is specified in the following way:

$$x_{i,j,k} = \begin{cases} 1 & \text{if the tile at coordinate } (i,j) \text{ takes on color } k \\ 0 & \text{otherwise} \end{cases}$$

Connectivity. Objective Function

Do we need an objective for the problem? **No.**

We care about the feasibility of the problem, meaning that all we need is a feasible coloring so that all flows are connected and do not intersect. Therefore, the objective function could simply be $\{\max\ 0\}$.

Connectivity. Constraints

Each tile must be colored with precisely one color, so we have

$$\sum_{k \in \texttt{COLORS}} x_{i,j,k} = 1 \qquad \text{for all } i \in \texttt{ROWS} \text{ and } j \in \texttt{COLUMNS}.$$

How should we enforce connectivity? To answer this question, we examine the sample solution in Figure 2 on the next page. The key observation is the following:

- If tile (i,j) is an `END-TILE` of color k, we need precisely one of its 4-neighbors to take on color k. This can be represented by the following pair of coupled constraints.

$$x_{i-1,j,k} + x_{i+1,j,k} + x_{i,j-1,k} + x_{i,j+1,k} \geq x_{i,j,k}$$
$$x_{i-1,j,k} + x_{i+1,j,k} + x_{i,j-1,k} + x_{i,j+1,k} \leq 4 - 3 \cdot x_{i,j,k}$$

  The smart thing about this formulation is that

  - When $x_{i,j,k} = 0$ (the (i,j) tile is not an `END-TILE` for color k), the constraints do not force anything.
  - When $x_{i,j,k} = 1$ (the (i,j) tile is an `END-TILE` for color k), the constraints force that precisely one of the 4-neighbors should also take on color k.

- If tile (i,j) is colored in color k but is not one of the `END-TILE`s (a `MIDDLE-TILE`) of color k, we need precisely two of its 4-neighbors to take on color k. In a similar fashion, we introduce the following pair of coupled constraints.

$$x_{i-1,j,k} + x_{i+1,j,k} + x_{i,j-1,k} + x_{i,j+1,k} \geq 2 \cdot x_{i,j,k}$$
$$x_{i-1,j,k} + x_{i+1,j,k} + x_{i,j-1,k} + x_{i,j+1,k} \leq 4 - 2 \cdot x_{i,j,k}$$

Example. Solutions to Flow!

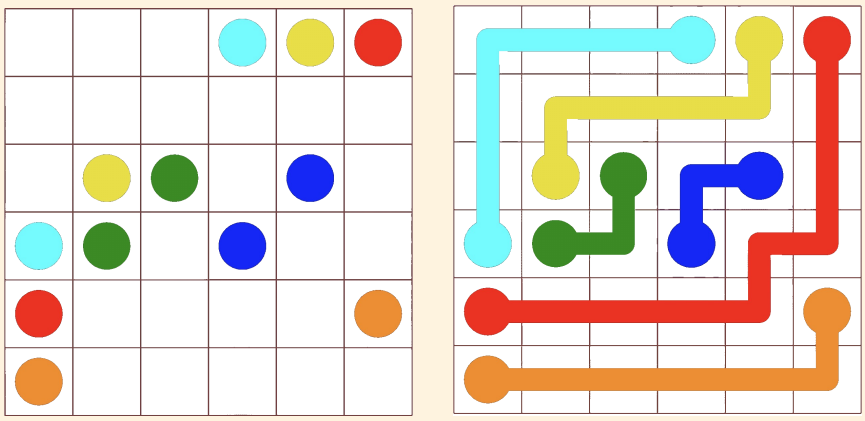


Figure 2: A new instance of Flow, and its corresponding solution as determined and visualized by our algorithm!

Recovering the Solution

After determining the feasible solution, we create a two-dimensional grid for $x_{i,j,k}$ for each color k. The $x_{i,j,k}$'s with value 1 will show the path!

Now that we know how to solve any arbitrary instance of Flow using LP connectivity, we will now consider a second formulation for solving Flow with LPs!!

Example. From 0's and 1's to Flows

```
[[1 1 1 1 1 1 1 1 1]
 [1 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 1 1 1]      ---->
 [1 0 0 0 0 0 1 0 0]
 [0 0 1 1 1 1 1 0 0]
 [0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0]]
```

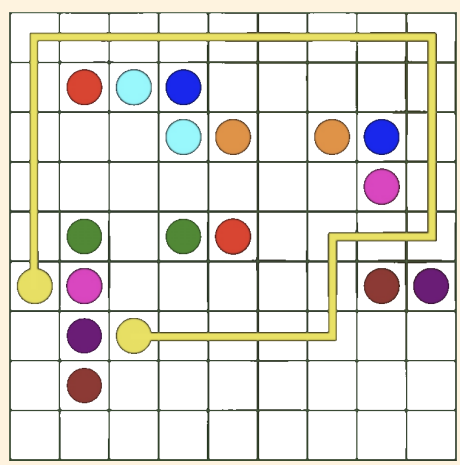
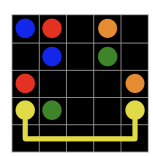


Figure 3: A representation of how we recover the flow for the color yellow!



Formulation 2: Reduction to the Multi-Commodity Flow Problem

We can treat each color k as a commodity. The two colored dots of color k that are initially on the board will play the roles of source and sink.

From Grid to Graph

The ultimate solution is the collection of several flows. We can perceive each flow k as a directed path along which we send commodity (color) k from its `SOURCE` to its `SINK`. If two tiles are next to each other, the flow could either go from Tile 1 to Tile 2, *or* the other way. We allow flows in both directions when we set up the graph. Consider Figure 4 on the right. For each tile, we construct

- outgoing edges from the tile to its neighbors, and
- incoming edges from its neighbors to the middle.

In addition, to ensure that there is only one color on each tile, we split the tile into two vertices, `in` and `out`, and construct an edge from its `in` vertex to its `out` vertex. All incoming edges arrive at the `in` vertex, and all outgoing edges depart from the `out` vertex.

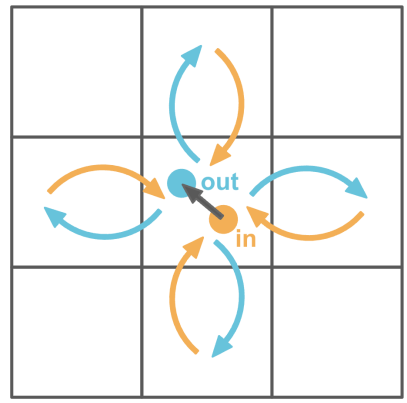


Figure 4: Graph Architecture

Encoding Vertices

We encode each vertex as

$$v = (\text{row index},\ \text{column index},\ \texttt{in}/\texttt{out}).$$

It is worth noting that if the (i, j)-th tile is a `SOURCE`, the source vertex is specifically the vertex $(i, j, \texttt{in})$. If the tile is instead a `SINK`, the sink vertex is specifically the vertex $(i, j, \texttt{out})$.

Multi-Commodity Flow. Decision Variables

Let $f_{u,v,k}$ denote the number of units of flow k that we push along the edge (u, v). We are not interested in the amount of flow we are able to push. Rather, we are interested in finding a *feasible* solution that sends flow k from its source to its sink. Furthermore, to avoid fractions in the final solution, we enforce that $f_{u,v,k}$ is binary.

Multi-Commodity Flow. Objective Function

Much like the connectivity formulation, this is also a feasibility problem. Thus, the objective is once again $\{\max\ 0\}$.

Multi-Commodity Flow. Constraints

We first introduce the *capacity constraint* that at most one unit of flow, regardless of its color, can be pushed along each edge. That is,

$$\sum_{k \in \texttt{COLORS}} f_{u,v,k} \leq 1 \qquad \text{for all } (u,v) \in \texttt{EDGES}.$$

Next, we introduce *flow balance constraints*. For each vertex v, the incoming and outgoing flows are given as follows

$$\text{Inflow}(v) = \sum_{\substack{k \in \texttt{COLORS}, \\ u \text{ where } (u,v) \in \texttt{EDGES}}} f_{u,v,k}, \qquad \text{Outflow}(v) = \sum_{\substack{k \in \texttt{COLORS}, \\ u \text{ where } (v,u) \in \texttt{EDGES}}} f_{v,u,k}.$$

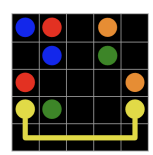
- For `SOURCE` vertices, we demand $\text{Outflow}(v) - \text{Inflow}(v) = 1$.
- For `SINK` vertices, we demand $\text{Inflow}(v) - \text{Outflow}(v) = 1$.
- For all `TRANSSHIPMENT` (intermediate) vertices, we demand $\text{Inflow}(v) - \text{Outflow}(v) = 0$.

Recovering the Solution

To recover the flows from the feasible solution, we create a two-dimensional grid for for the $f_{u,v,k}$ values for each color k. If $f_{u,v,k} = 1$, color k is sent from u to v, so tiles that represent u and v should both take on the color k. In particular, if

$$u = (i_1, j_1, \texttt{out}) \quad \text{and} \quad v = (i_2, j_2, \texttt{in}),$$

then tiles (i_1, j_1) and (i_2, j_2) should both take on color k. It is possible that u and v happened to be the `in` and `out` vertices of the same tile, in which case we are just specifying twice that the tile should take on color k.



Summary of the Linear Programs

We summarize the two linear programs that we have formulated as follows:

Linear Program. Connectivity!

$$
\begin{aligned}
\max \quad & 0 \\
\text{s.t.} \quad & \sum_{k \in \texttt{COLOR}} x_{i,j,k} = 1 && \text{for all } i \in \texttt{ROWS} \text{ and } j \in \texttt{COLUMNS} \\
& x_{i-1,j,k} + x_{i+1,j,k} + x_{i,j-1,k} + x_{i,j+1,k} \;\geq\; x_{i,j,k} && \text{for all } (i,j) \in \texttt{END-TILES} \\
& && \quad \text{and } k \in \texttt{COLORS} \\
& x_{i-1,j,k} + x_{i+1,j,k} + x_{i,j-1,k} + x_{i,j+1,k} \;\leq\; 4 - 3x_{i,j,k} && \text{for all } (i,j) \in \texttt{END-TILES} \\
& && \quad \text{and } k \in \texttt{COLORS} \\
& x_{i-1,j,k} + x_{i+1,j,k} + x_{i,j-1,k} + x_{i,j+1,k} \;\geq\; 2x_{i,j,k} && \text{for all } (i,j) \in \texttt{MIDDLE-TILES} \\
& && \quad \text{and } k \in \texttt{COLORS} \\
& x_{i-1,j,k} + x_{i+1,j,k} + x_{i,j-1,k} + x_{i,j+1,k} \;\leq\; 4 - 2x_{i,j,k} && \text{for all } (i,j) \in \texttt{MIDDLE-TILES} \\
& && \quad \text{and } k \in \texttt{COLORS} \\
& x_{i,j,k} \text{ binary}
\end{aligned}
$$

Linear Program. Multi-Commodity Flow!

$$
\begin{aligned}
\max \quad & 0 \\
\text{s.t.} \quad & \sum_{k \in \texttt{COLORS}} f_{u,v,k} \leq 1 && \text{for all } (u,v) \in \texttt{EDGES} \\
& \sum_{\substack{k \in \texttt{COLORS} \\ v \text{ where } (u,v) \in \texttt{EDGES}}} f_{u,v,k} - \sum_{\substack{k \in \texttt{COLORS} \\ v \text{ where } (v,u) \in \texttt{EDGES}}} f_{v,u,k} = 1 && \text{for all } u \in \texttt{SOURCES} \\
& \sum_{\substack{k \in \texttt{COLORS} \\ v \text{ where } (v,u) \in \texttt{EDGES}}} f_{v,u,k} - \sum_{\substack{k \in \texttt{COLORS} \\ v \text{ where } (u,v) \in \texttt{EDGES}}} f_{u,v,k} = 1 && \text{for all } u \in \texttt{SINKS} \\
& \sum_{\substack{k \in \texttt{COLORS} \\ v \text{ where } (u,v) \in \texttt{EDGES}}} f_{u,v,k} - \sum_{\substack{k \in \texttt{COLORS} \\ v \text{ where } (v,u) \in \texttt{EDGES}}} f_{v,u,k} = 0 && \text{for all } u \in \texttt{TRANSSHIPMENT} \\
& f_{u,v,k} \text{ binary}
\end{aligned}
$$

Runtime "Analysis"

Let the problem instance have R rows, C columns, and K colors. Then, roughly speaking,

- The Connectivity formulation has RCK decision variables and $2RCK$ functional constraints.
- The Multi-Commodity Flow formulation has $4RCK$ decision variables and $6RC$ functional constraints.

In real-life, the Connectivity formulation generally runs faster than the Multi-Commodity Flow formulation, as the Multi-Commodity Flow formulation contains more terms in its constraints and requires more time to recover the solution from the results.

Further Discussions

It turned out that both formulations are quite generalizable! Below we discuss several variants of Flow instances which the algorithms can be easily modified to solve.

Variant 1: What if there are walls?

Note: Flows cannot pass through walls.

For the *Connectivity formulation*, we enforce that the candidate neighbors can only be chosen from tiles that are in direct contact with the middle tile, i.e., there is no wall in between.

For the *Multi-Commodity Flow formulation*, we remove the edges that go across the wall.

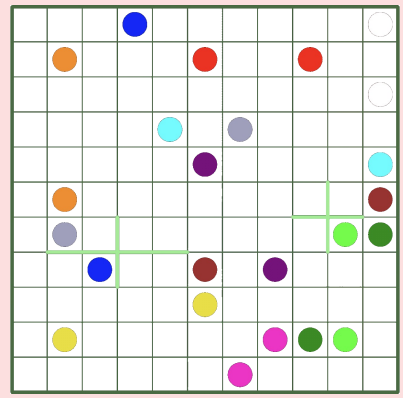


Figure 5: Flow instance with walls

Variant 2: What if the space is irregular?

For the *Connectivity formulation*, we enforce that the candidate neighbors can only be chosen from tiles that are within the space and in direct contact with the middle tile. Note that there could be up to six adjacent tiles as seen in the hexagonal instance.

For the *Multi-Commodity Flow formulation*, we create edges only across two neighbor tiles that are both present in the space.

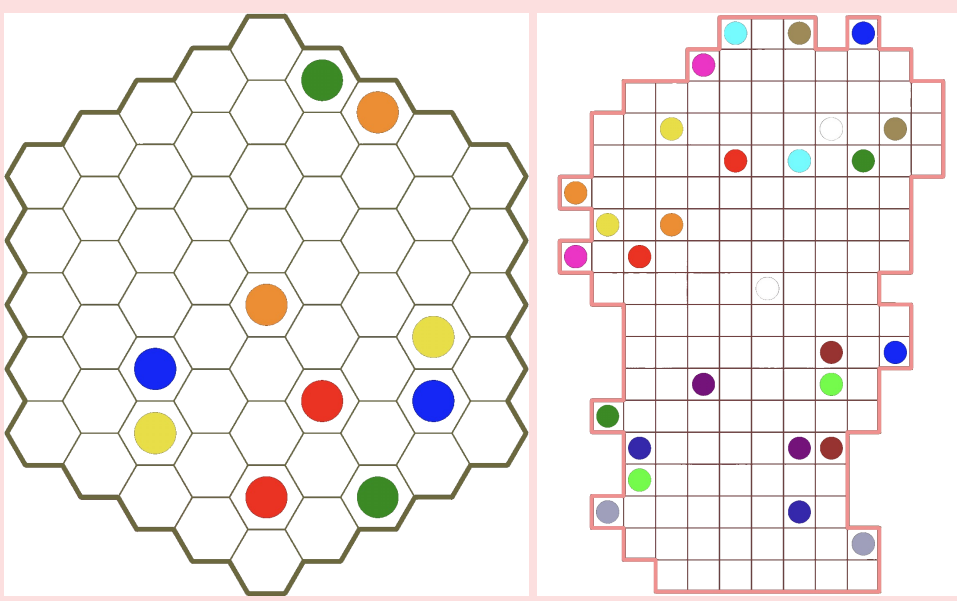


Figure 6: Flow instances where the shape of the tiles and/or the board is irregular

Variant 3: What if flows can warp (a "Pacman" situation)?

Note: For each tile on the boundary with a dashed line, a flow can pass through the boundary and is teleported into the corresponding "Pacman" tile located on the other side (if the "Pacman" tile is not occupied by another colored dot).

For the *Connectivity formulation*, for each boundary tile, we define its "Pacman" tile to be one of its candidate 4-neighbors.

For the *Multi-Commodity Flow formulation*, we add edges that go from each boundary tile to the corresponding "Pacman" tile.

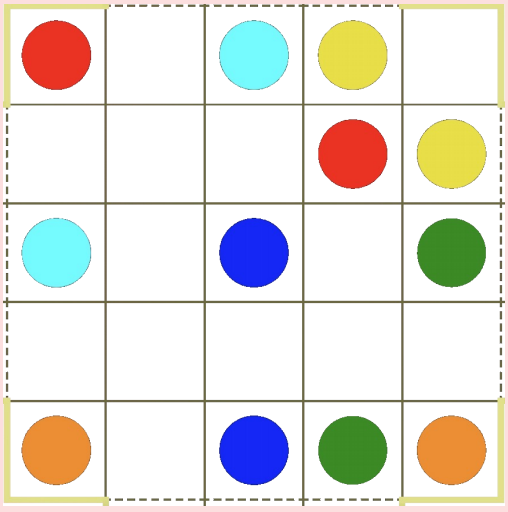


Figure 7: Flow instance with warping

Variant 4: What if there are bridges?

Note: Flows cannot turn at a bridge. However, two flows are permitted to cross at a bridge tile.

For the *Connectivity formulation*, for each color k at a bridge tile, we require its two neighbors to be either the North-South pair or the West-East pair, representing the fact that flows cannot turn at bridge tiles. Also, $\sum_k x_{i,j,k} = 2$ for each bridge tile with coordinate (i, j), representing the fact that two flows will cross at the bridge.

For the *Multi-Commodity Flow formulation*, we skip the edge construction for each bridge tile, and bidirectionally connect its North-South pair and West-East pair of neighbors. To recover the coloring at the bridge tile, we look at the two colors that the 4-neighbors of the bridge tile take, and color the bridge tile in those two colors.

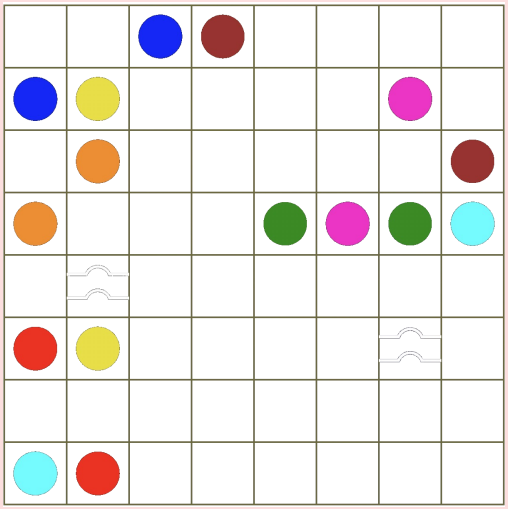
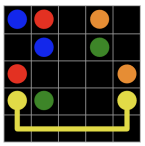
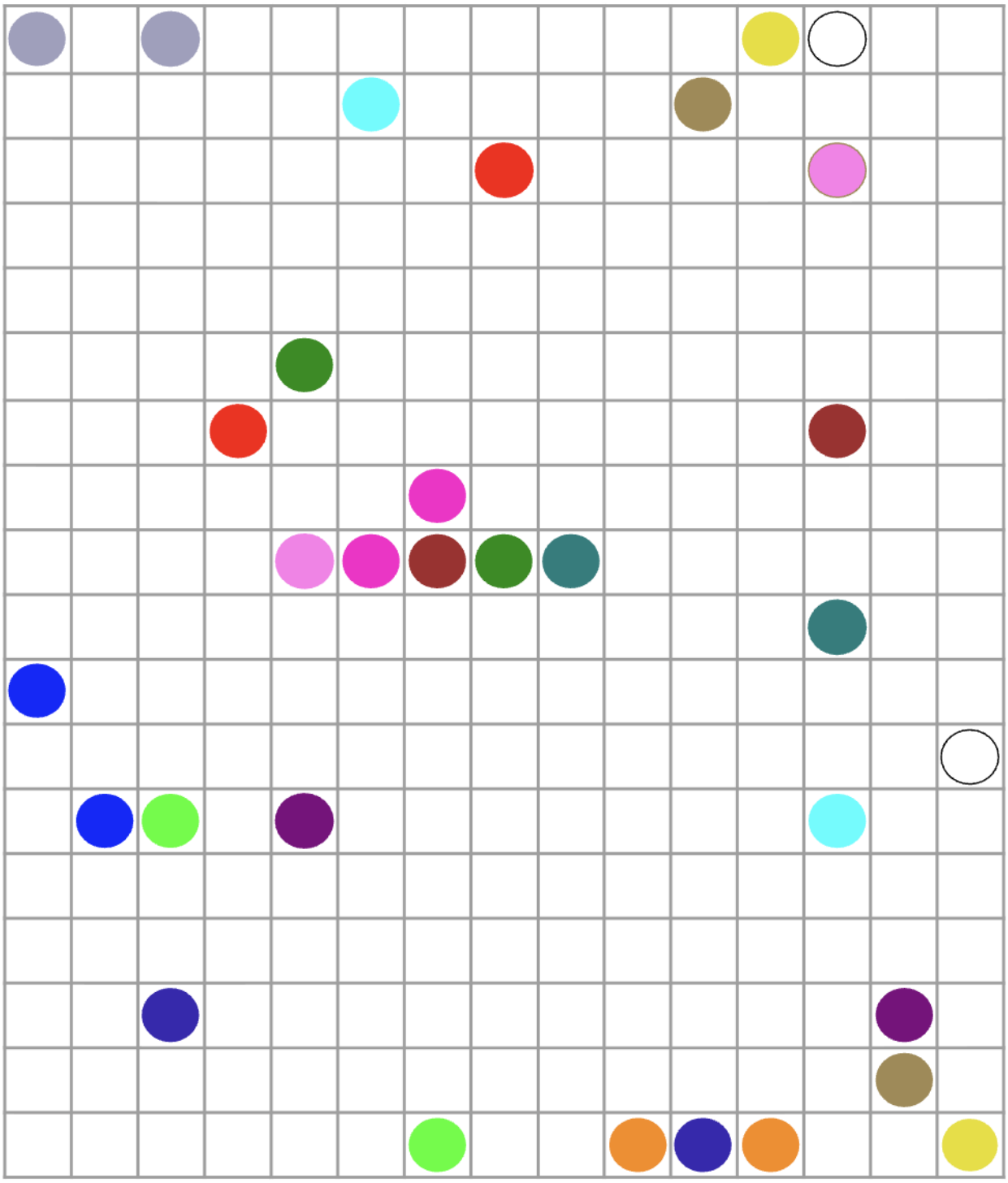


Figure 8: Flow instance with bridges

Note: We purposefully left these problems blank—see if you can solve them yourself!



Another hard problem – have fun!



Note: If you are the first person to solve this problem before class ends, *Tommy Liu* will buy you a drink of your choice!!